

Respuestas Cuestionario PLENITAS - Alex Tumiri Huanca

Perfil Profesional: B.Sc. en Computer Science (USFX) | Candidato a M.Sc. en Inteligencia Artificial (UNIR). Especialista en Backend (Java/Spring Boot/Quarkus) e Ingeniería de IA.

Bloque 1: Soft skills y autonomía

P1: Cuéntame una situación donde tuviste que tomar una decisión técnica importante sin consultar a nadie. ¿Qué decidiste y cómo resultó? historia concreta

Durante mi trayectoria en **JITKey (España)**, identifiqué que el sistema de predicción de precios enfrentaba latencias que afectaban la experiencia del usuario. Decidí, de manera autónoma, implementar una arquitectura de microservicios utilizando **Quarkus** sobre **AWS Serverless**. Esta elección fue estratégica debido a su bajo consumo de memoria y rápido tiempo de respuesta. El resultado fue una optimización del 40% en la velocidad del API de pricing, permitiendo actualizaciones de tarifas en tiempo real sin degradar el rendimiento del BackOffice.

P2: ¿Cómo sueles integrarte a un equipo o proyecto nuevo? ¿Qué haces en las primeras semanas? proceso concreto

Mi proceso de integración se basa en la observación técnica y el aporte de valor inmediato. Durante las primeras dos semanas, audito la documentación en **Confluence** y analizo el historial de despliegues en **GitLab o BitBucket**. Me gusta identificar rápidamente "puntos de dolor" técnicos. Generalmente, realizo mis primeros Pull Requests en tareas de refactorización o corrección de bugs menores durante la primera semana para validar mi comprensión de los estándares de calidad del equipo sin introducir riesgos operativos.

P3: ¿Has liderado o mentoreado a otros desarrolladores? Cuéntame cómo fue. experiencia real de liderazgo

En roles Senior en **Niubiz (Perú)** y **Prisma (Argentina)**, he actuado como referente técnico. Mi enfoque de mentoría se centra en *Code Reviews* constructivos y sesiones de *Pair Programming*. En Prisma, lideré la refactorización de librerías de encriptación críticas, guiando a desarrolladores junior en la implementación de **Spring WebFlux** y programación reactiva, asegurando que el equipo no solo entregara código funcional, sino que comprendiera los principios de escalabilidad y seguridad bancaria.

Bloque 2: Filtro técnico básico

P4: Explícame como si fuera un no-técnico: ¿qué es un monolito y qué es un microservicio? ¿Cuándo tiene sentido migrar de uno a otro?.

Imagina una cocina de restaurante. Un **monolito** es un solo chef que se encarga de todo: tomar el pedido, cocinar, servir y lavar. Si el restaurante se llena, el chef se satura y todo el servicio se detiene. Un **microservicio** es tener especialistas: un pizzero, un experto en postres y un camarero coordinados. Si hay muchos pedidos de pizza, solo el pizzero necesita ayuda, mientras los demás siguen trabajando normalmente. Tiene sentido migrar cuando el negocio crece tanto que un solo equipo no puede gestionar todo el código de forma rápida, o cuando partes específicas necesitan una potencia de cómputo diferente al resto.

P5: ¿Con qué tecnologías has trabajado más en los últimos 2 años? ¿En qué proyecto te sientes más orgulloso/a del backend que desarrollaste?

En los últimos dos años, mi foco ha estado en el ecosistema **Java (Spring Boot, Quarkus)**, arquitecturas de eventos con **Kafka** e integración de **IA Generativa (LangChain, Modelos LLM)**. Me siento especialmente orgulloso del motor de ingesta y conciliación de transacciones que desarrollé en **Niubiz**. Fue un reto de alta disponibilidad donde procesamos volúmenes masivos de datos financieros en tiempo real sobre **AWS**, garantizando una precisión del 100% en los reportes para entidades bancarias.

P6: ¿Usas herramientas de IA para desarrollar? ¿Cuáles y cómo las integras en tu día a día?

Utilizó la IA como un multiplicador de productividad. En el día a día, empleo **GitHub Copilot** para automatizar código boilerplate y generar pruebas unitarias robustas (JUnit/Mockito). Además, debido a mi formación en IA, integro herramientas como **LangSmith y LangGraph** para diseñar y monitorear agentes inteligentes que automatizan procesos de backend, asegurando que las respuestas de los modelos sean seguras y eficientes antes de llegar a producción.

Bloque 3: Defensa de la prueba técnica

P7: Explícame las 2-3 decisiones de diseño más importantes de tu prueba. ¿Por qué las tomaste así?

Decisión 1: Arquitectura Multi-Plataforma con Modelo UserPlatform Separado

Por qué la tomé así:

- Seguridad: Permite credenciales diferentes por plataforma para el mismo email
- Integridad de Datos: Relaciones explícitas con claves foráneas aseguran consistencia
- Escalabilidad: Fácil agregar campos específicos por plataforma (last_login, preferencias)
- Rendimiento: Consultas eficientes sin joins complejos

Alternativas descartadas:

- **Campo JSON en User:** Pérdida de integridad referencial y consultas complejas
- **Relación many-to-many:** No permite credenciales específicas por plataforma

Decisión 2: Sistema de Reglas Configurables con JSONField

Por qué la tomé así:

Flexibilidad: Nuevas reglas sin cambios en schema

Runtime: Configurable vía admin interface

Performance: JSONField indexable en PostgreSQL/SQLite moderno

Simplicidad: Fácil de entender y mantener

Alternativas descartadas:

- **Modelo separado para reglas:** Over-engineering para key-value simple
- **Columnas por regla:** Requeriría migraciones para nuevas reglas

Decisión 3: JWT con Contexto de Plataforma Integrado

Por qué la tomé así:

Stateless: No requiere almacenamiento de sesiones

Performance: Elimina lookups de BD en cada request

Seguridad: Contexto de plataforma previene acceso cross-platform

Escalabilidad: Soporta horizontal scaling

Alternativas descartadas:

- Tokens separados por plataforma: Complejidad y overhead
- Sesión tradicional: No escala bien para APIs

**P8: ¿Cómo implementaste las reglas configurables por plataforma?
¿Qué alternativas consideraste?**

Implementación Realizada:

```
```python
class Platform(models.Model):
 business_rules = models.JSONField(default=dict)

 def get_rule(self, rule_name, default=None):
 return self.business_rules.get(rule_name, default)

 def set_rule(self, rule_name, value):
 self.business_rules[rule_name] = value
 self.save()
...
```
```

Estructura de Reglas por Defecto:

```
```python
{
 'max_devices': 5,
 'allow_inactive_login': True,
 'device_name_required': True,
 'ip_validation_enabled': False,
 'session_timeout_hours': 24
}
```
```

Validación en Tiempo Real:

```
```python
def create(self, validated_data):
 platform = user_platform.platform
 max_devices = platform.get_rule('max_devices', 5)

 if not user_platform.can_add_device():
 raise serializers.ValidationError(
 f"Límite de dispositivos ({max_devices}) alcanzado"
)
```
```

Alternativas Consideradas:

1. Modelo Rule por Plataforma ❌

- **Pro:** Fuertemente tipado
- **Contra:** Over-engineering, complejidad excesiva

2. Archivo de Configuración Python ✖

- **Pro:** Versionable
- **Contra:** Requiere deployments para cambios

3. Redis para Reglas ✖

- **Pro:** Alto performance
- **Contra:** Dependencia adicional, complejidad deployment

4. Columnas de BD por Regla ✖

- **Pro:** Tipado fuerte
- **Contra:** Migraciones requeridas para nuevas reglas

Por qué JSONField es la mejor opción:

- Balance perfecto de flexibilidad y performance
- Sin cambios de schema para nuevas reglas
- Configurable en runtime vía admin
- Queryable y indexable

P9: ¿Qué cambiarías de tu solución si tuvieras 1 semana más?

1. Implementación de Microservicio FastAPI para Devices

Por qué:

- **Performance:** FastAPI es ~2-3x más rápido que Django DRF
- **Async:** Soporte nativo async/await para I/O operations
- **Aislamiento:** Domain boundary claro para device management

Implementación:

```
```python
devices_service/
|— main.py # FastAPI app
|— models.py # SQLAlchemy models
|— schemas.py # Pydantic schemas
|— api/devices.py # Device endpoints
|— core/security.py # JWT validation
...
```
```

Migración Gradual:

1. Extraer lógica de devices
2. Implementar FastAPI con mismos contratos
3. Database replication o event streaming
4. Load balancer para routing gradual

2. Sistema de Caching Redis Avanzado

Implementación:

```
```python
Cache strategy
CACHE_KEYS = {
 'platform_rules': 'platform:{id}:rules',
 'user_devices': 'user:{user_id}:platform:{platform_id}:devices',
 'user_platform': 'user:{user_id}:platform:{platform_id}'
}
```

# Cache TTL por tipo

CACHE\_TTL = {

'platform\_rules': 3600, # 1 hora

'user\_devices': 300, # 5 minutos

'user\_platform': 1800 # 30 minutos

}

...

### **Beneficios:**

- Reducción de queries BD 60-80%
- Response times < 50ms para endpoints cacheados
- Menos load en base de datos

## **3. Sistema de Eventos con Celery**

### **Arquitectura de Eventos:**

```
```python
```

```
# Event-driven architecture
```

```
@celery.task
```

```
def handle_device_created(device_id, user_platform_id):
```

```
    # Send notifications
```

```
    # Update analytics
```

```
    # Check platform limits
```

```
    # Trigger webhooks
```

```
@celery.task
```



```
def handle_user_registered(user_platform_id):  
    # Welcome email  
    # Default device creation  
    # Analytics tracking  
    ...
```

Eventos Implementados:

- `user\_registered`
- `device\_added/removed`
- `login\_success/failed`
- `platform\_created`

4. Testing Automatizado CI/CD

GitHub Actions Workflow:

```
```yaml
```

```
name: CI/CD Pipeline
```

```
on: [push, pull_request]
```

```
jobs:
```

```
 test:
```

```
 runs-on: ubuntu-latest
```

```
 services:
```

```
 postgres: latest
```

```
 redis: latest
```

```
 steps:
```

```
 - python: 3.11
```

```
 - coverage: 95%+
```

- integration tests
- security scan

...

## 5. Monitoring y Observabilidad

### Implementación:

```
```python
# Prometheus metrics

from prometheus_client import Counter, Histogram

DEVICE_REQUESTS = Counter('device_requests_total', 'Total device requests')
REQUEST_DURATION = Histogram('request_duration_seconds', 'Request duration')

# Structured logging
import structlog
logger = structlog.get_logger()
...

```

6. Documentación API Automatizada

****OpenAPI/Swagger Avanzado:****

```
```python
Enhanced API docs

@router.post(
 "/devices/",
 response_model=DeviceResponse,

```

```

summary="Create new device",
description="Create a device with platform-specific validation",
responses={
 201: {"description": "Device created successfully"},
 400: {"description": "Validation error or device limit exceeded"},
 401: {"description": "Authentication required"}
}
)
...

```

## 7. Security Enhancements

### Implementaciones:

```

```python
# Rate limiting
from django_ratelimit import ratelimit

@ratelimit(key='ip', rate='100/h', method='POST')
def create_device(request):
    # Device creation logic

# IP whitelisting by platform
def validate_ip_platform(ip_address, platform):
    allowed_ranges = platform.get_rule('allowed_ip_ranges', [])
    return ip_in_ranges(ip_address, allowed_ranges)
...

```

8. Database Optimizations

****Índices Específicos:****

```sql

-- Performance indexes

CREATE INDEX idx\_user\_platform\_user\_active

ON accounts\_user\_platform(user\_id, is\_active);

CREATE INDEX idx\_device\_user\_platform\_active

ON devices\_device(user\_platform\_id, is\_active);

CREATE INDEX idx\_notification\_recipient\_read

ON notifications\_notification(recipient\_id, is\_read);

...

### **9. Docker Compose Production-Ready**

```yaml

version: '3.8'

services:

web:

image: backend:latest

replicas: 3

environment:

- DATABASE_URL=postgresql://...

- REDIS_URL=redis://redis:6379

- CELERY_BROKER_URL=redis://redis:6379

postgres:

image: postgres:15

environment:

POSTGRES_DB: backend_api

redis:

image: redis:7-alpine

nginx:

image: nginx:alpine

ports:

- "80:80"

- "443:443"

...

Prioridades de Implementación (1 semana):

1. **Día 1-2:** Caching Redis básico y optimización queries
2. **Día 3-4:** Sistema de eventos Celery
3. **Día 5:** Microservicio FastAPI para devices
4. **Día 6:** CI/CD pipeline y testing automatizado
5. **Día 7:** Monitoring y documentación avanzada

Impacto Esperado:

- **Performance:** 50-70% mejora en response times
- **Scalability:** Soporte para 10x más usuarios

- **Reliability:** 99.9% uptime con monitoring
- **Maintainability:** Testing automatizado y CI/CD

Estas mejoras transformarían la solución de prototipo funcional a un sistema production-ready listo para escalar a millones de usuarios.

Evaluación de Inglés

P10: Experience with Django REST Framework

*"While my professional core has been focused on **Java/Spring** and **Node.js**, I have a strong **Python** background through my Master's in AI, where I work extensively with PyTorch and TensorFlow. I am fully familiar with RESTful principles and ORM patterns. I have used Django patterns for AI-driven projects, and I can quickly adapt my deep backend architecture knowledge to deliver high-quality APIs using **Django REST Framework** by leveraging my experience in software design and robust infrastructure."*